
DeckForge AI

A locally fine-tuned diffusion model for skateboard decals,
on 8 GB of VRAM

Kylian Algoet · Multimedia & Creative Technologies

Erasmushogeschool Brussel · Bachelor defence, 2 September 2026

The assignment, and the question under it

A skateboard manufacturer wants **customer-designed decks** — without a designer in the loop for every order.

How can a **locally fine-tuned** diffusion model, conditioned on **text and a reference image**, generate decal artwork in **multiple visually distinct styles** with **reproducible quality** on **8 GB of VRAM**?

docs/00-project-brief.md · thirteen mandatory requirements · RQ1–RQ12 · assessed on the research process, not the artefact

One constraint, and how the work was done

8187.5 MiB on the card. Under real serving the device holds **7987.5 MiB** at worst, leaving **200 MiB** spare — **2.4 % of the card.**

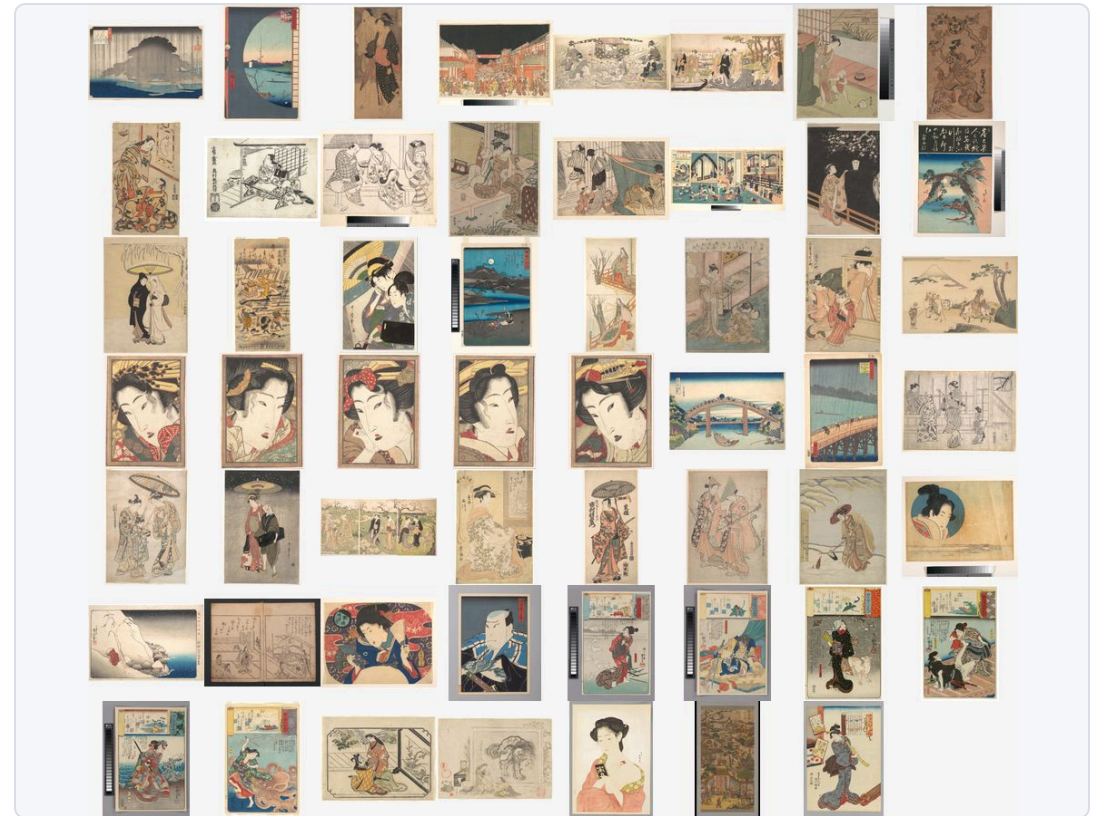


Each prototype answers the question the next one depends on. None was skipped.

40 experiments · 17 decision records · two human approval gates against a rubric defined before the images were reviewed — **the first blinded, the second labelled by necessity.**

The dataset was built, not downloaded

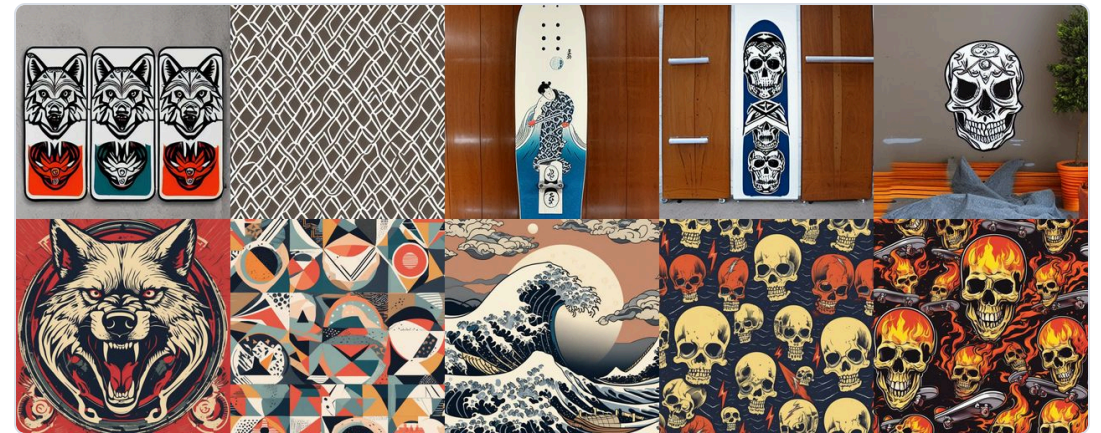
- **148 items**, three styles, built for this project — holdout of 7 never seen in training
- Every item is **CC0, public domain, or made for this project**. No third-party licensed work
- **Licence and source URL recorded before the item entered a split** — a gate, not an afterwards audit
- Captions are **style-only**. That was tested against verbatim descriptions, and style-only won



Ukiyo-e split. Metropolitan Museum of Art, CC0. Every item carries its licence and source URL in the manifest.

SDXL was better, and was rejected

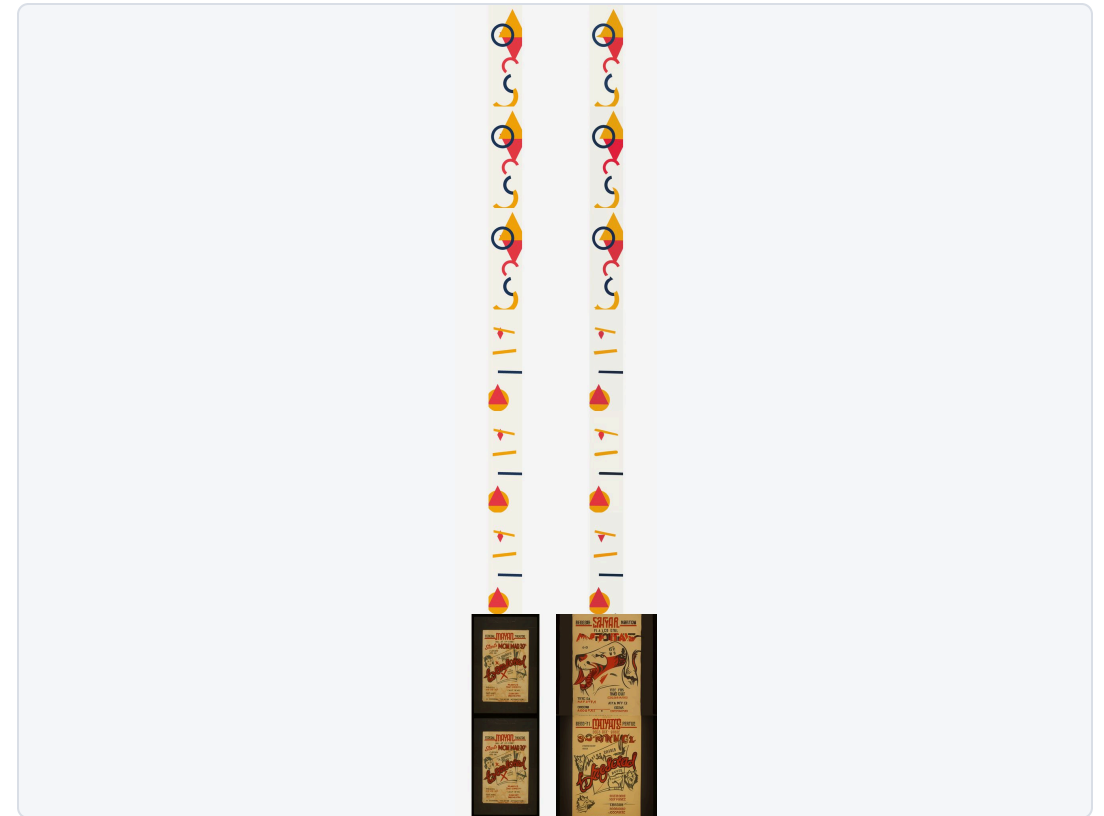
- SDXL scored **better** than SD 1.5 on my own rubric, at its native resolution
- Rejected anyway: **10 738 MiB allocated** at 1024^2 , on a 8187.5 MiB card
- Windows **spilled into host memory instead of failing**. All thirty runs reported success
- SD 1.5 selected — not because it wins, but because **it is the one that fits at a useful resolution**



Track B — 1024×1024, SDXL's designed resolution. The images the 10 738 MiB measurement came from.

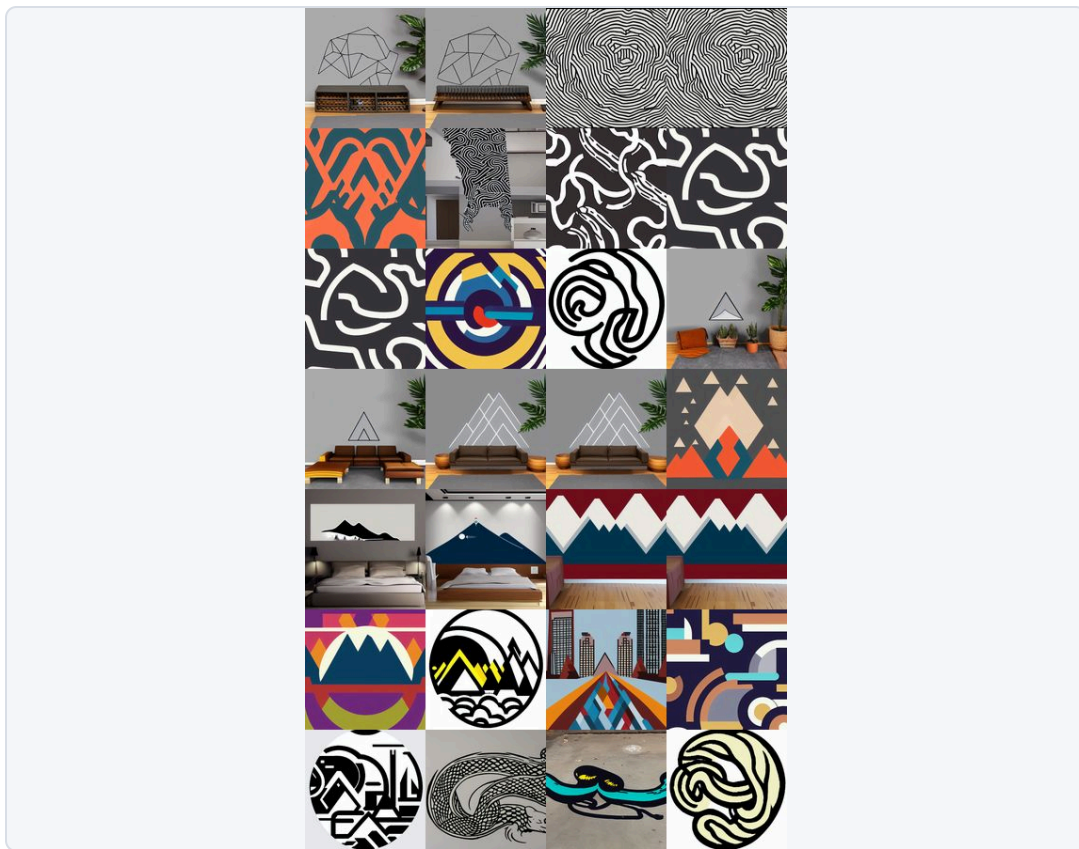
The free method lost

- Two ways in for a reference image: **img2img** or **IP-Adapter**
- img2img costs **zero extra VRAM** — decisive on this budget, and it lost anyway
- At the deck format it **returned its own input**. Every near-copy flag came from img2img
- **IP-Adapter selected**, scale 0.55. A method that reproduces its input is not a generator.

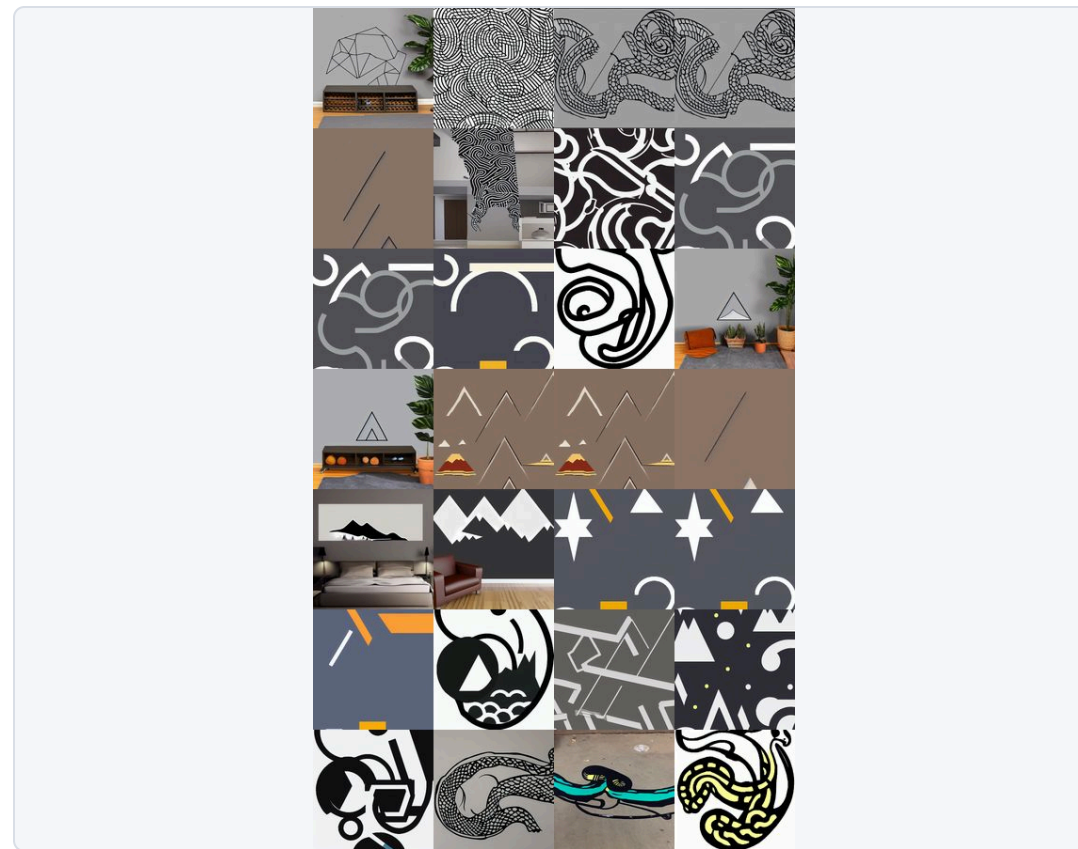


Reference, then img2img output. Some pairs are perceptually indistinguishable. This is why the free method was not selected.

Training longer made it less obedient



Step 300 — shipped. Style consistency 5, prompt adherence 4. EXP-027.

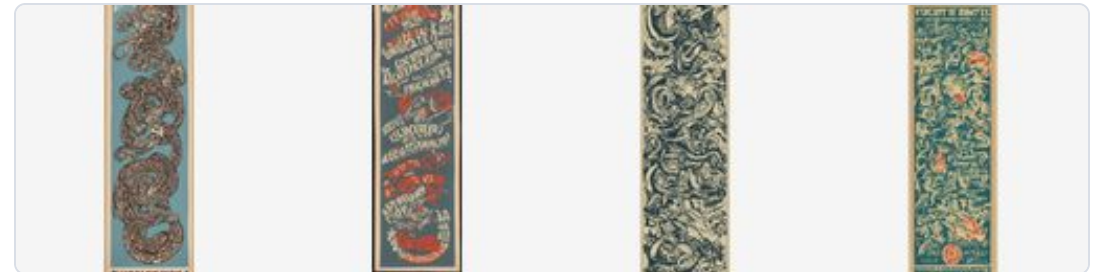


Step 600 — rejected. Style still 5, prompt adherence 3. Same prompts, same seeds.

Longer training bought style and spent obedience. Three per-style LoRA adapters, rank 8, applied at 0.7 — two ship at step 300, ukiyo-e at 600. **The claim is feasibility, not superiority.**

What did not work

- **retro-poster ships as a PARTIAL PASS** — it learned the posters' frames and lettering. Labelled in the interface; not dropped, not re-scored until it passed
- **How many images does a style need? INCONCLUSIVE** — 12, 24 and 44, non-monotonic, one run each
- **No second independent human rater** — one human approver plus AI-assisted visual analysis, so no inter-rater agreement
- SD 2.1 was authentication-gated; memorisation is **bounded, not settled**



EXP-029, step 300. The frames and lettering are inherited from the source material, not requested by the prompt.

The integrated system

Generation goes **direct to 512×1536** — not square-then-crop. **One worker is asserted in code**, because a second resident pipeline does not fit: a concurrent request gets a clean refusal, never an out-of-memory crash.

```
<rect x="404" y="40" width="360" height="200" rx="14" fill="#f4f6f9" stroke="#d9dee6" stroke-width="1" />
<text x="428" y="80" font-size="24" font-weight="700" fill="#11151c">FastAPI service</text>
<text x="428" y="116" font-size="21" fill="#46525f">uvicorn, <tspan font-weight="700" fill="#11151c">worker</tspan>
<text x="428" y="148" font-size="21" fill="#46525f">Busy lock &#8594; 409, never OOM</text>
<text x="428" y="180" font-size="21" fill="#46525f">Upload validation, timeouts</text>
<text x="428" y="206" font-size="21" fill="#46525f">SHA-256 checkpoint verify</text>
```

```
<rect x="834" y="40" width="380" height="200" rx="14" fill="#11151c" />
<text x="858" y="80" font-size="24" font-weight="700" fill="#ffffff">One resident pipeline</text>
<text x="858" y="116" font-size="21" fill="#c8d0dc">Stable Diffusion 1.5</text>
<text x="858" y="148" font-size="21" fill="#c8d0dc">+ one per-style LoRA</text>
<text x="858" y="180" font-size="21" fill="#c8d0dc">+ IP-Adapter (optional)</text>
```

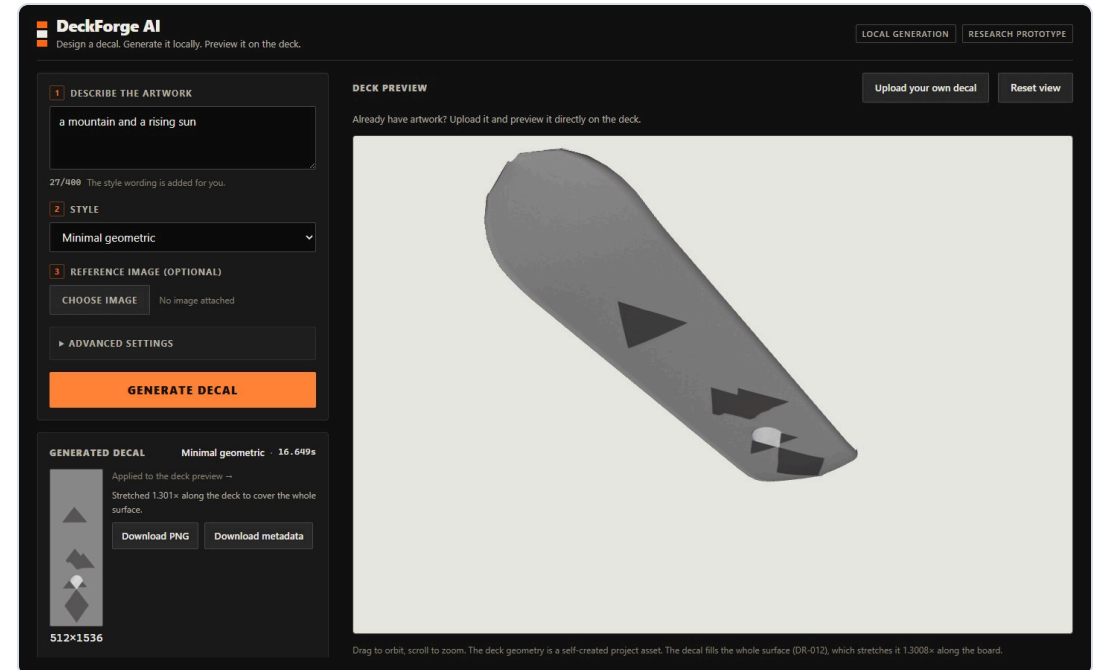
Testing, and what it cannot tell you

- **473** product and research tests + **16** report-validation tests = **489 pytest** · 183 vitest · 38 end-to-end
- The **clean-clone test** is the one that counts: empty directory, install from scratch, working system
- It caught an **integrity control that had only ever passed on the machine that wrote it**
- **A green suite does not prove the generator makes good pictures.** Only the human gates did that

[docs/evidence/M8/clean-clone/](#) · report §15

Reproducible has halves

- **Inference is deterministic.** A clean clone reproduced an earlier output **byte for byte**, three days later, in a fresh environment
- **Training is not.** Adapter initialisation draws from an **unseeded global generator** — a run cannot be repeated from its recorded seed
- So the checkpoints are **artifacts, verified by hash on every request** — not recipes
- A defect that produces perfectly valid-looking runs, and is **invisible unless you compare weights**



Clean clone, fresh environment. SHA-256

46bbf160e4270429e6692467dc6c59577e99bf3178dedd8d38193d0335fb6d7f,
identical to the original run.

Local, and chosen

- Four options compared on nine criteria. **Selected: native local run + pre-generated backup assets**
- **200 MiB spare prevents a second resident pipeline**, so multi-worker inference is unsupported
- Docker GPU passthrough was **never verified on this machine**; unmeasured overhead is disqualifying on this margin
- **Public cloud was deliberately not selected** — cost, a demo-day network dependency, and no research gain
- Reproducibility rests on the **runbook and the clean-clone proof**, not on a public URL

DR-014 · alternatives A–D · EXP-034

Conclusions

Yes — and the configuration that fits is specific and measured. SD 1.5 at 512×1536, one per-style LoRA at 0.7, IP-Adapter at 0.55.

Bounded three ways: **the margin is 2.4 % · reproducible describes inference, not training · three styles, and one is partial.**

Not concluded: that LoRA beats the alternatives · that SD 1.5 is the best base model · that any image count is a threshold · that a green suite means good pictures.

Live demo

Prompt → style → **512×1536 decal** → 3D deck. **Generation runs locally on this laptop's GPU; no cloud inference is used.**

If the GPU does not cooperate, that is a live-demo problem and not a project problem — there is a recorded run ready and I will say which one you are looking at.

What I would change

- **Repeat every condition before drawing a curve.** The image-count question is inconclusive because I ran each condition once — a planning mistake, not a resource one
- **Seed everything, then verify by comparing weights** rather than trusting the log
- **Get a second rater.** One approver cannot produce inter-rater agreement, and I could have arranged one
- **40 experiments · 17 decision records · six prototypes** — and conclusions that stop where the evidence stops

github.com/KylianAlgoet/Selftrained-and-deployed-AI-image-generator · docs/learning-outcome-traceability.md